Student 21 **Lakshmi Narayanan E** 192524249RD 5 / 5 submitted

Q1. Exception

Score: 10.00 **TC: 3/3 passed**

CODE

```
import java.util.*;

class Main{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int arr[] = new int [5];
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;
        int in;
        in = sc.nextInt();
        try{
            System.out.println(arr[in]);
        }
        catch(ArrayIndexOutOfBoundsException ar){
            System.out.println("Exception: Invalid array index.");
        }
    }
}
```

INPUT

```
6
```

OUTPUT

```
Exception: Invalid array index.
```

Q2. IllegalArgumentException

Score: 10.00 **TC: 3/3 passed**

CODE

```
import java.util.Scanner;
class illegalArgument {
    public static void main(String args[]) {
        int n;
        Scanner s = new Scanner(System.in);
        n = s.nextInt();
        try {
            if (n < 0 || n > 100) {
                throw new IllegalArgumentException("Invalid marks. Marks should be between 0 and 100.");
            }
            System.out.println(n);
        }
        catch (IllegalArgumentException ar) {
            System.out.println("Exception: " + ar.getMessage());
        }
    }
}
```

INPUT

```
120
```

OUTPUT

```
Exception: Invalid marks. Marks should be between 0 and 100.
```

Q3. user-defined exception named WeakPasswordException.

Score: 0.00 **TC: 0/3 passed**

CODE

```
import java.util.Scanner;
class WeakPasswordException extends Exception{
    WeakPasswordException(String a)
    {
        super(a);
    }
    static void check(String password) throws WeakPasswordException{
        if(password.length()<8){
            throw new WeakPasswordException("Password must contain at least 8 characters.");
        }
    }
    public static void main(String args[]){
        Scanner s = new Scanner(System.in);
        String password = s.nextLine();
        try{
            check(password);
        }
        catch(WeakPasswordException e){
            System.out.println(e.getMessage());
        }
    }
}
```

INPUT

```
java123
```

OUTPUT

```
Password must contain at least 8 characters.
```

Q4. Email Validation

Score: 0.00 **TC: 0/3 passed**

CODE

```
import java.util.Scanner;
    public class InvalidEmailException extends Exception{
        InvalidEmailException(String a)
        {
            super(a);
        }
        static void check(String mail)throws InvalidEmailException{
            if(!mail.contains("@")){
                throw new InvalidEmailException("Invalid email address");
            }
            else{
                System.out.println("valid");
            }
        }
        public static void main(String args[]){
            String mail;
            Scanner s = new Scanner(System.in);
            mail=s.nextLine();
            try{
                check(mail);
            }
            catch(InvalidEmailException e){
                System.out.print(e.getMessage());
            }
        }
    }
```

INPUT

```
student.gmail.com
```

OUTPUT

```
Invalid email address
```

Q5. Banking Transaction Using Exception Propagation

CODE

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        sc.next();
        int balance = sc.nextInt();

        sc.next();
        int withdrawAmount = sc.nextInt();

        try {
            withdraw(balance, withdrawAmount);
        }
        catch (InsufficientBalanceException e) {
            System.out.println("Exception propagated to main().");
            System.out.println("InsufficientBalanceException: " + e.getMessage());
        }
    }

    static void withdraw(int balance, int withdrawAmount)
            throws InsufficientBalanceException {

        validateBalance(balance, withdrawAmount);
    }

    static void validateBalance(int balance, int withdrawAmount)
            throws InsufficientBalanceException {

        processTransaction(balance, withdrawAmount);
    }

    static void processTransaction(int balance, int withdrawAmount)
            throws InsufficientBalanceException {

        if (withdrawAmount > balance) {
            throw new InsufficientBalanceException(
                "Withdrawal amount exceeds available balance."
            );
        }
    }
}

class InsufficientBalanceException extends Exception {

    InsufficientBalanceException(String message) {
        super(message);
    }
}
```

OUTPUT

```
(no output)
```